

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.11

PAGINATION

 Copy as Markdown

Pagination

Serverpod provides built-in support for pagination to help manage large datasets, allowing you to retrieve data in smaller chunks. Pagination is achieved using the `limit` and `offset` parameters.

Limit

The `limit` parameter specifies the maximum number of records to return from the query. This is equivalent to the number of rows on a page.

```
var companies = await Company.db.find(  
  session,  
  limit: 10,  
);
```

In the example we fetch the first 10 companies.

Offset

The `offset` parameter determines the starting point from which to retrieve records. It essentially skips the first `n` records.

```
var companies = await Company.db.find(  
  session,  
  limit: 10,  
  offset: 30,  
);
```

In the example we skip the first 30 rows and fetch the 31st to 40th company.



Using limit and offset for pagination

Together, `limit` and `offset` can be used to implement pagination.

```
int page = 3;
int companiesPerPage = 10;

var companies = await Company.db.find(
  session,
  orderBy: (t) => t.id,
  limit: companiesPerPage,
  offset: (page - 1) * companiesPerPage,
);
```

In the example we fetch the third page of companies, with 10 companies per page.

Tips

1. **Performance:** Be aware that while `offset` can help in pagination, it may not be the most efficient way for very large datasets. Using an indexed column to filter results can sometimes be more performant.
2. **Consistency:** Due to possible data changes between paginated requests (like additions or deletions), the order of results might vary. It's recommended to use an `orderBy` parameter to ensure consistency across paginated results.
3. **Page numbering:** Page numbers usually start from 1. Adjust the offset calculation accordingly.

Cursor-based pagination

A limit-offset pagination may not be the best solution if the table is changed frequently and rows are added or removed between requests.

Cursor-based pagination is an alternative method to the traditional limit-offset pagination. Instead of using an arbitrary offset to skip records, cursor-based pagination uses a unique record identifier (a *cursor*) to mark the starting or ending point of a dataset. This approach is particularly beneficial for large datasets as it offers consistent and efficient paginated results, even if the data is being updated frequently.

How it works

In cursor-based pagination, the client provides a cursor as a reference point, and the server returns data relative to that cursor. This cursor is usually an `id`.

Implementing cursor-based pagination

1. **Initial request:** For the initial request, where no cursor is provided, retrieve the first `n` records:

```
int recordsPerPage = 10;
```

```
var companies = await Company.db.find(
  session,
  orderBy: (t) => t.id,
  limit: recordsPerPage,
);
```

2. **Subsequent requests:** For the subsequent requests, use the cursor (for example, the last `id` from the previous result, typically sent by the client) to fetch the next set of records:

```
int cursor = lastCompanyIdFromPreviousPage;

var companies = await Company.db.find(
  session,
  where: (t) => t.id > cursor,
  orderBy: (t) => t.id,
  limit: recordsPerPage,
);
```

3. **Returning the cursor:** When returning data to the client, also return the last row's `id`, so the client can send it back as the cursor for the next page. Define a small model that carries the page of rows together with its cursor, for example a `CompanyPage` with a `List<Company>` field and a nullable `lastCursor` field.

Tips

1. **Choosing a cursor:** While IDs are commonly used as cursors, timestamps or other unique, sequentially ordered fields can also serve as effective cursors.
2. **Backward pagination:** To implement backward pagination, use the first item from the current page as the cursor and adjust the query accordingly.
3. **Combining with sorting:** Ensure the field used as a cursor aligns with the sorting order. For instance, if you're sorting data by a timestamp in descending order, the cursor should also be based on the timestamp.
4. **End of data:** If the returned data contains fewer items than the requested limit, it indicates that you've reached the end of the dataset.